

Spark vs. Ray : Assignment 3

DA3408 Assignment 3
Aramati Chiru Thejaswi (DA24B036)

October 4, 2026

Summary

In this assignment, I implemented the same data cleaning pipeline using PySpark and Ray Data. I ran both frameworks on a two-worker setup using 20 NYC Yellow Taxi Parquet files from January 2025 to August 2026.

The input data was about 1.3 GB in compressed form. Both frameworks produced exactly 57,068,386 rows, and I checked the outputs value by value.

For the final comparable runs, Spark took 223.89 seconds and Ray took 248.58 seconds. Spark was also able to complete the heavier operations involving shuffling, while Ray could not complete those operations on the 5.8 GiB machine.

So, for this experiment, **Spark was the winner**. However, the timing comparison is based on only one run for each framework.

1 Setup

The following table shows the setup used for Spark and Ray.

Item	Spark	Ray
Version	Apache Spark 4.2.0 (standalone)	Ray 2.58.0, Ray Data
Cluster	1 master + 2 workers, with 2 cores and 1 GB memory per worker	1 head node with 0 CPU + 2 worker nodes with 2 CPUs each
Machine	One VirtualBox VM with 4 cores, 5.8 GiB RAM and 4 GiB swap	Same VM
Data	20 Parquet files and taxi_zone_lookup.csv with 265 zones	Same data

Table 1: Experimental setup

Both clusters were run on the same VM, one at a time. Therefore, the workers were actually sharing the same physical machine. There was also no real network between the workers.

For Spark, the Master UI was on port 8080 and the application UI was on port 4040. For Ray, the dashboard was on port 8265.

2 Pipeline

I kept the main pipeline the same for both frameworks.

2.1 1. Reading the data

I read all 20 Parquet files and selected the same 9 columns. Some files from June 2026 onwards had one extra column called `request_source`. Selecting the columns explicitly kept the schemas the same.

2.2 2. Cleaning the data

I removed rows with null values and kept only rows satisfying these conditions:

- Drop-off time is after pickup time.
- Trip duration is at most 6 hours.
- Distance is greater than 0 and at most 100 miles.
- Number of passengers is at least 1.

I also created:

- `duration_sec`
- `pickup_date`
- `pickup_hour`

The optional exact de-duplication step was controlled using a `--dedup` flag.

2.3 3. Transforming the data

I performed two inner joins with the taxi zone table:

- pickup location with the zone table
- drop-off location with the zone table

I also calculated average speed using the following formula:

$$avg_speed_mph = round\left(\frac{trip_distance}{duration_sec/3600}, 2\right)$$

2.4 4. Export

The final output was written as Parquet files.

Ray wrote about 8.5 GiB of output data, and both frameworks produced the same number of rows.

3 Pipeline Parity

I used a DuckDB script to compare the Spark and Ray output folders without loading all the data into memory.

The main results are shown below.

Check	Spark	Ray	Result
Row count	57,068,386	57,068,386	Match
Number of columns	17	17	Match
Nulls, minimum and maximum values	Same	Same	Match
Whole-row checksum	52636231323400069450720	52636231323400069450720	Match

Table 2: Spark and Ray output comparison

The floating-point column sums differed only by very small rounding differences. The relative difference was below 10^{-9} . This is expected because the two systems can add floating-point values in a different order.

The row-level comparison was exact.

3.1 Data quality observations

I also noticed some interesting issues in the original data:

- One month had 540,149 rows out of 3,475,226 rows with nulls in five columns at the same time. This is about 15.5%.
- Some trips had drop-off times before pickup times.
- Some trips had zero distance.
- Some distances were extremely large, up to 276,423 miles.
- Some pickup dates went back to 2007. These rows were not removed because the assignment specification did not include a date filter.
- No exact duplicate rows were found.

4 Execution Time

The wall-clock time includes reading the data, processing it, and writing the output.

For the final comparison, both frameworks used the same basic pipeline, with no de-duplication.

Run	Time	Rows
Spark cluster	223.89 s	57,068,386
Ray cluster	248.58 s	57,068,386

Table 3: Final execution times

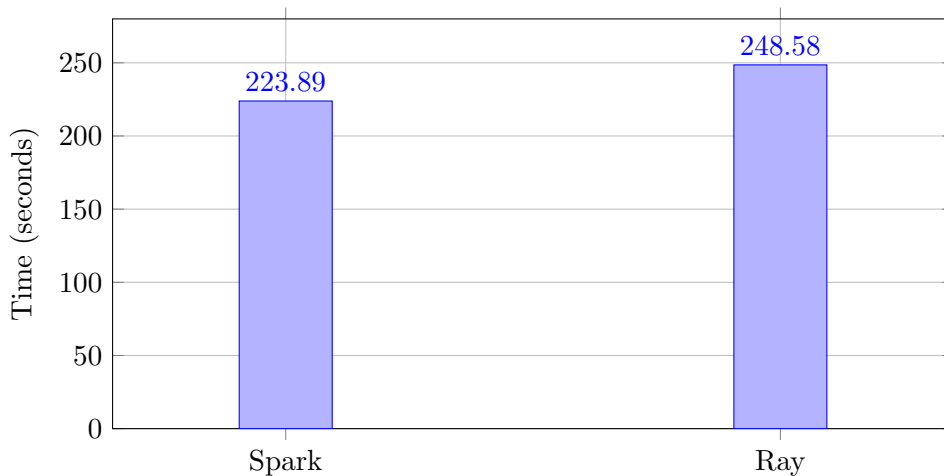


Figure 1: Final execution time comparison

Spark was 24.7 seconds faster than Ray. This means Spark took about 10% less time, while Ray took about 11% longer than Spark.

There were also some earlier Spark runs with different configurations. I do not use them as the final comparison because the configuration was changed between runs.

One Spark run took 475.92 seconds because another copy of the script was accidentally running at the same time and competing for CPU and memory.

The Spark time later dropped from 332.08 seconds to 223.89 seconds after removing de-duplication and using a broadcast join. Since both changes were made together, I cannot say how much each individual change helped.

5 Shuffle-Heavy Operations in Ray

The first Ray implementation used a hash-shuffle join and a global group-by for de-duplication.

On the 5.8 GiB VM, Ray used almost all of the available memory. The dashboard showed about 3.9 out of 3.9 GiB of memory in use, and the object store was using about 470 out of 477 MiB.

The group-by reached only about 10% after around 12 minutes. At that point, the estimated remaining time was about 1 hour 25 minutes, so I stopped it.

A later hash-join attempt caused repeated out-of-memory worker kills.

In comparison, Spark completed the equivalent de-duplication and shuffle join in 332.08 seconds on the same machine.

This is a useful observation from the experiment, but it is not a fully controlled benchmark.

6 Resource Utilization

The following observations came from the framework dashboards.

For the final Spark run:

- There were 2 executors.
- Each executor had 2 cores.
- There were 37 tasks.
- No tasks failed.
- Garbage collection took about 21 seconds out of about 24 minutes of total task time, which is around 1.5%.

For an earlier Ray attempt:

- CPU usage was about 82% on each of the three nodes.
- The head node object store used about 286 MB.
- Each worker object store used about 477 MB.

I did not collect clean peak CPU and peak memory logs for the final runs. Therefore, I did not include a direct peak-utilization comparison.

7 Python UDF Overhead

I did not measure the UDF overhead separately. Both frameworks ran the UDF as part of a larger processing stage, so the exact cost could not be read from the dashboards.

The UDF calculates average speed using simple arithmetic.

In Spark, a Python UDF runs in a separate Python process. Data has to move between the JVM and Python, which creates serialization overhead. Spark also treats the Python UDF as a black box.

In Ray, `map_batches` runs the function directly inside a Ray worker process using pandas or Arrow. There is no JVM-to-Python boundary.

Because of this design, Ray can be attractive for workloads with heavier Python code, such as feature engineering or model inference.

However, in this experiment the UDF was very simple, and Spark was still faster overall. Therefore, the Python advantage of Ray was not visible in this particular workload.

8 Performance Tuning

I used AI assistance while developing the pipeline. An LLM helped with some pipeline logic and suggested some performance changes. The cluster setup, network setup, and actual timings were done manually.

The main changes were:

1. **Ray join:** I changed the hash-shuffle join to a map-side join. The small 265-row zone table was loaded once and used for each batch. This avoided a large shuffle.
2. **Spark join:** I used a broadcast hint for the same small zone table and removed the setting that forced shuffle joins.
3. **De-duplication:** I made de-duplication optional using the `--dedup` flag because the global group-by was not practical for Ray on this machine.
4. **Schema/parity fix:** I read the zone CSV in a way that kept the literal text “N/A” as a string in both frameworks. I also made timestamp and zone ID types consistent.

9 Limitations

There are several limitations in this experiment.

- Each final timing is based on only one run. Therefore, I cannot measure normal run-to-run variation.
- Both frameworks were run on one VM. The workers shared the same CPU, memory and disk, so this is not the same as a real multi-machine cluster.
- The operating system file cache may have been warmer for later runs. The order of the runs was also not randomized.
- Peak CPU and memory usage were not measured cleanly for both final runs.
- The UDF overhead was not measured separately.

Because of these limitations, the 24.7-second difference should not be treated as a universal benchmark result.

10 Conclusion

For this experiment, **Apache Spark was the better choice.**

Spark completed the final pipeline in 223.89 seconds, while Ray took 248.58 seconds. Spark was also able to complete the shuffle-heavy operations that Ray could not finish on the 5.8 GiB machine.

However, this does not mean Spark is always better than Ray. The result depends on the type of project.

BI-first project	AI-first project
Recommendation: Spark Spark is a good choice for structured and SQL-style data processing, joins, aggregations and large shuffle operations. It also remained stable under the memory limits in this experiment.	Recommendation: Ray Ray can be useful for Python-heavy pipelines where the processed data is directly used for model training or inference. Its Python-native <code>map_batches</code> can also work well with GPU-based workloads.

Table 4: Framework recommendation by project type

For this particular workload, the measured results support Spark. The possible Python advantage of Ray is based on the architecture and was not directly measured in this experiment.

11 Evidence

During the final Spark run, the Spark Master UI showed 2 workers alive and all 4 worker cores being used. The application was running as `spark_clean`.

The Ray dashboard showed 3 nodes alive: 1 head node and 2 worker nodes.